

[BOJ] 13422(C++/cpp)

☑ 미완	☐
📄 언어	C++
≡ 문제 출제	BOJ
✨ 진행 상황	성공
≡ 알고리즘 분류	투 포인터
☑ 복습 필요	☒
📅 풀이날	@2025년 3월 12일
📄 난도	골드 4
☑ 발전 가능성 여부	☐

문제

n 개의 집이 원형으로 배치되어 있으며, 연속된 m 개의 집을 골라 훔칠 수 있습니다. 단, 훔친 금액의 합이 k 미만이어야 합니다.

각 테스트 케이스마다 가능한 경우의 수를 구해야 합니다.

풀이

아이디어

- 슬라이딩 윈도우 사용
 - 연속된 m 개의 부분 배열을 빠르게 탐색하기 위해 **슬라이딩 윈도우 기법**을 사용합니다.
 - 초기 윈도우(처음 m 개의 합)를 구한 후, 한 칸씩 이동하면서 윈도우를 업데이트합니다.
- 원형 배열 처리
 - m 개의 연속된 집을 검사하는 과정에서 인덱스가 끝을 넘어갈 수 있음.
 - `houses[right % n]` 을 사용하여 **원형 배열처럼 순회**합니다.

- 예외 처리 (`n == m`)

- 이 경우, 가능한 부분 배열은 오직 한 개이므로 한 번만 검사 후 반환합니다.

코드

```
#include <iostream>
#include <vector>

using namespace std;

int oneRound() {
    int n, m, k;
    cin >> n >> m >> k;

    vector<int> houses(n);

    // 입력 받기
    for (int i = 0; i < n; i++) {
        cin >> houses[i];
    }

    int ans = 0, sum = 0;

    // 초기 윈도우 설정
    for (int right = 0; right < m; right++) {
        sum += houses[right];
    }

    if (sum < k) {
        ans++;
    }

    // 특별 케이스: n == m이면 가능한 부분 배열은 하나뿐이므로 바로 반환
    if (n == m) {
        return ans;
    }

    // 슬라이딩 윈도우 실행
```

```

int left = 0, right = m;

while (right < n + m - 1) {
    sum -= houses[left % n];
    sum += houses[right % n];

    if (sum < k) {
        ans++;
    }

    left++;
    right++;
}

return ans;
}

int main() {
    int t;
    cin >> t;

    for (int i = 0; i < t; i++) {
        cout << oneRound() << "\n";
    }

    return 0;
}

```

해결 과정에서 어려웠던 점

- 예외 처리 (`n == m`) 누락
 - 처음에는 `n == m` 인 경우도 슬라이딩 윈도우를 실행하려 했지만, 이 경우 가능한 부분 배열이 오직 하나뿐이므로 한 번만 검사 후 반환해야 함을 깨달음.
- 슬라이딩 윈도우의 `right` 초기값 설정 실수
 - `right = m - 1` 로 설정하면 첫 번째 윈도우 크기가 잘못됨.
 - `right = m` 으로 설정하여, `right++` 시 정확히 다음 원소를 가리키도록 조정.

발전 가능한 아이디어